

Rithmic Trial to HftBacktest Pipeline Prompt

Developer-ready integration prompt for temporary Rithmic trial data capture, validation, and replay wiring.

Task: Wire the temporary Rithmic trial data feed into the existing HftBacktest pipeline.

Objective:

Add a temporary Rithmic trial ingestion lane so live paper/trial market data can be captured, normalized, validated, converted, and replayed through the existing HftBacktest workflow.

This is a wiring and integration task only. Do not redesign the trading system. Do not replace the existing backtester. Do not change production strategy logic. Do not pollute the main data lake.

Before coding:

1. Inspect the current repo structure.
2. Identify the existing data-ingestion layer, event schema, HftBacktest converter, validation/reporting utilities, and storage conventions.
3. Reuse existing components wherever possible.
4. If an equivalent component already exists, extend it cleanly instead of creating a duplicate.
5. Document exactly where the Rithmic trial lane plugs into the current pipeline.

Required pipeline flow:

Rithmic trial / paper environment / available bridge

- > raw append-only capture
- > normalized internal event schema
- > data-quality validation
- > HftBacktest-compatible replay files
- > replay/conversion reports
- > existing research/backtest workflow

Scope:

Use the currently available Rithmic trial environment while formal API approval is pending. The connector may use R|Trader Pro, an approved platform bridge, export mechanism, or any currently available trial-compatible feed path, but the implementation must clearly separate the temporary bridge from the permanent future Rithmic API connector.

Hard rules:

1. Do not mutate raw captured data.
2. Do not write Rithmic trial data into the trusted production lake.
3. Do not mark trial data as production-approved.
4. Do not bypass existing validation gates.
5. Do not create a parallel backtesting system.
6. Do not hard-code symbols, contracts, paths, timestamps, or session assumptions.
7. Do not assume the trial feed exposes every event type; detect and report what is actually available.
8. Preserve compatibility with future direct Rithmic API ingestion.

Capture requirements:

Capture every event type available from the trial connection, including where available:

- trades
- best bid/ask quotes
- market depth updates
- order submissions
- order acknowledgments
- fills

- cancels
- rejects
- position/account updates if exposed

Each event must include, where available:

- source = rithmic_trial
- capture_environment = paper_or_trial
- symbol
- contract
- exchange
- event_type
- exchange_timestamp
- local_receive_timestamp_ns
- local_write_timestamp_ns
- price
- size
- side
- bid levels
- ask levels
- order_id if applicable
- fill_id if applicable
- sequence number if available
- gateway/session metadata if available

Storage requirements:

Create a quarantined storage lane such as:

```
data/raw/rithmic_trial_live_capture/YYYY-MM-DD/SYMBOL/
data/normalized/rithmic_trial_live_capture/YYYY-MM-DD/SYMBOL/
data/replay/hftbacktest/rithmic_trial/YYYY-MM-DD/SYMBOL/
reports/rithmic_trial/YYYY-MM-DD/
```

All raw files must be append-only and accompanied by:

- manifest.json
- schema_version
- row/event counts
- first_timestamp
- last_timestamp
- checksum
- capture_start_time
- capture_end_time
- detected_event_types
- missing_event_types
- known limitations

Normalization requirements:

Map raw Rithmic trial events into the existing internal market-event schema. If the existing schema is insufficient, add a minimal backward-compatible extension with versioning. Do not break existing data consumers.

Validation requirements:

Add or reuse validation checks for:

- missing timestamps
- duplicate events
- out-of-order events
- crossed books
- locked books
- negative or zero prices
- negative sizes
- impossible spreads
- missing depth resets
- session gaps
- symbol/contract mismatch

- timezone errors
- local timestamp monotonicity
- malformed order/fill events
- unsupported event types

HftBacktest requirements:

Add a converter from the normalized Rithmic trial format into the existing HftBacktest-compatible replay format.

The converter must:

1. Preserve event ordering.
2. Preserve local receive timestamps.
3. Preserve exchange timestamps when available.
4. Preserve trade and depth information separately.
5. Produce one replayable output per symbol/session.
6. Emit a conversion report.
7. Fail loudly if required fields are missing instead of silently producing bad replay files.

Reports required:

Produce the following reports automatically:

```
reports/rithmic_trial/YYYY-MM-DD/data_capture_report.json
reports/rithmic_trial/YYYY-MM-DD/schema_mapping_report.json
reports/rithmic_trial/YYYY-MM-DD/data_quality_report.json
reports/rithmic_trial/YYYY-MM-DD/book_reconstruction_report.json
reports/rithmic_trial/YYYY-MM-DD/latency_profile.json
reports/rithmic_trial/YYYY-MM-DD/hftbacktest_conversion_report.json
```

Each report must clearly state:

- pass/fail status
- event counts
- missing fields
- detected limitations
- warnings
- exact input files
- exact output files
- schema versions used

Acceptance criteria:

The task is complete only when:

1. The Rithmic trial feed can be connected from the colocated server or documented trial-compatible bridge.
2. Raw live trial data is captured append-only.
3. Raw data is quarantined from the production lake.
4. Normalized event files are produced.
5. Validation reports are generated.
6. HftBacktest-compatible replay files are produced.
7. A sample replay can run through the existing HftBacktest workflow.
8. The implementation documents exactly which event types are available from the trial path.
9. The implementation documents exactly what must be swapped once official Rithmic API access is approved.
10. Existing backtest/research workflows still run without regression.

Future API compatibility:

Design the temporary Rithmic trial bridge behind a connector interface so the future direct Rithmic API connector can replace only the input adapter.

The rest must remain unchanged:

- raw schema
- normalized schema
- validation checks
- report format
- HftBacktest converter
- storage layout
- research/backtest integration

Deliverables:

1. Code changes implementing the temporary Rithmic trial ingestion lane.
2. Config file for enabling/disabling the Rithmic trial source.
3. Storage layout documentation.
4. Schema mapping documentation.
5. Validation/report examples.
6. One working sample capture-to-HftBacktest replay run.
7. Clear notes on limitations of the trial feed.
8. Clear handoff notes for replacing the temporary bridge with the official Rithmic API connector later.

Important:

This must be production-style wiring, even though the data source is temporary. The purpose is to get live Rithmic trial data into the existing HftBacktest pipeline safely, cleanly, and reversibly without confusing the main Quant X architecture.